

Introduction: We will use the cache to optimize data handling in our java CPU simulator. The aim is to reduce clock cycles using our java classes instructionCache (L1) and L2Cache (L2), which simulate the L1 and L2 caches combined.

Analysis: Using our assembly test programs, we managed to get a total cycle of 879,444 for un-implemented cache, 211,364 for instruction cache only, and 162,694 with instruction cache and L2 cache combined. Our assembly programs test for numbers starting from 0 to 100 and add them with a basic loop, array and a linked list, and titled as (LoopSum, ArraySum and LinkedListSum). The clock cycles according to cache implementations and classes are shown below:

Architecture	LoopSum	ArraySum	LinkedListSum	Total Cycles
Un-cached	61,708	303,716	514,020	879,444
L1 Cache Only	3,138	70,866	137,360	211,364
L1 & L2 Combined	3,208	53,406	106,080	162,694

The percent difference of L1 with non-cache = $\frac{879,444-211,364}{879,444} \times 100 = \%75.97$, and the percent difference of L1 & L2 combined with non-cache = $\frac{879,444-162,604}{879,444} \times 100 = 81.51\%$.

To further highlight these architectural differences, we can also prove that array is faster than a linked list during the summation, while the looping is the fastest method and defeats them all. The reason for that is, looping only uses registers in our test program. For the array and linked list programs, we accessed memory with store and load, which in fact resulted in way more clock cycles. The reason linked list resulted in approximately %50 more clock cycles than array is that the linked nodes store an additional pointer in the memory. Therefore, when selecting a data structure for basic sequential operations, arrays are the vastly superior and more hardware-efficient choice.

Conclusion: Outcome of the experiment is that we optimized and reduced the clock cycles at most by %81.51 using the successfully simulated l1 and l2 caches combined.